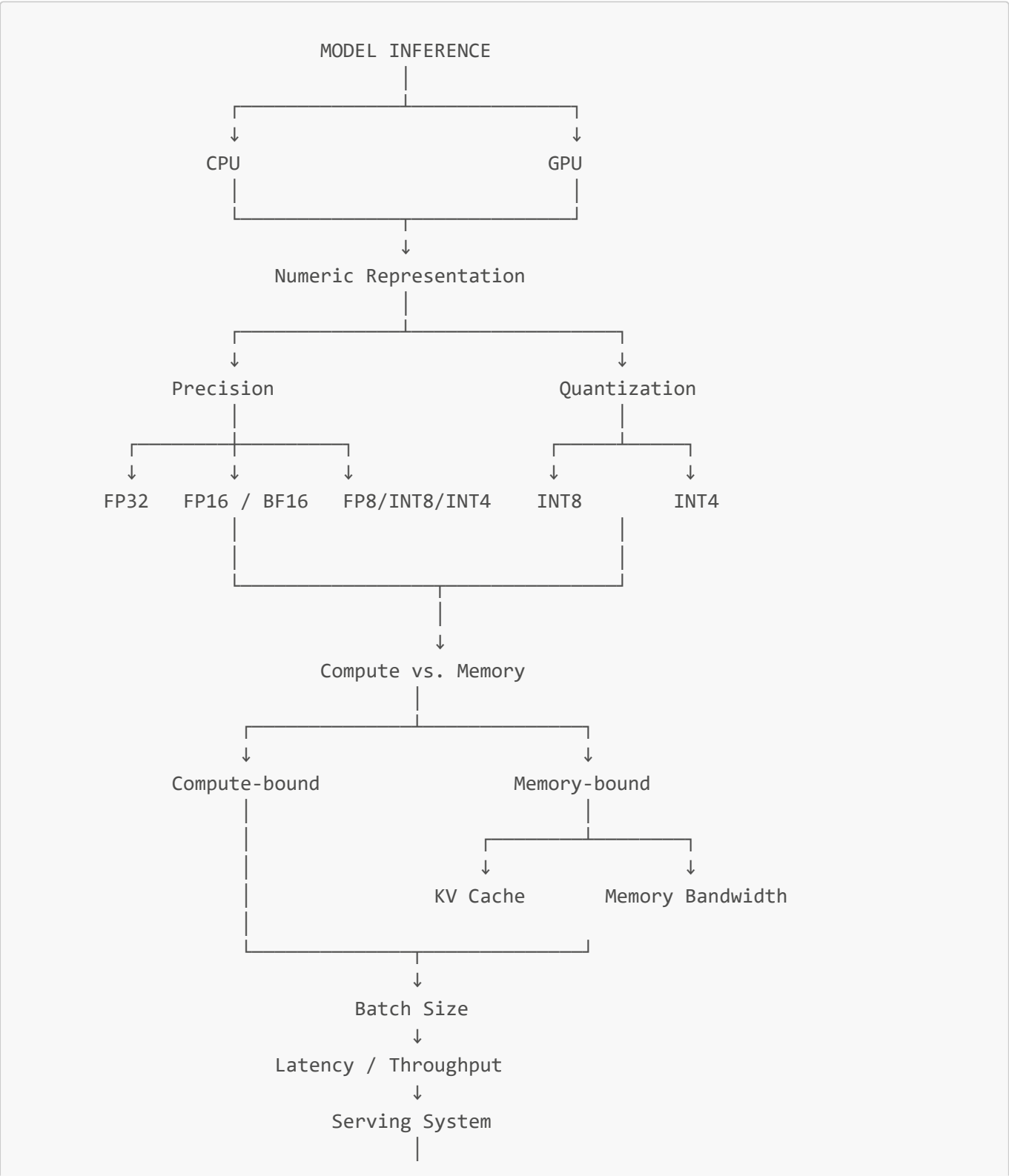
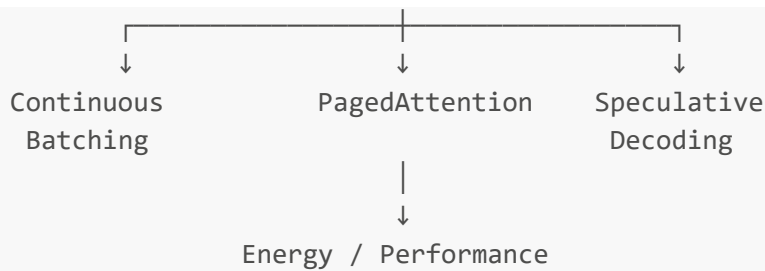


Deep Dive: Hardware Execution, Floating-Point Precision & Energy Dynamics in Model Inference

本文件系統化整理 AI 模型 **Inference** 過程中的 **GPU/CPU Execution Model**、**Floating-Point Precision & Hardware Design Philosophy**、**Quantization**、**Memory vs. Compute Bottlenecks**、**Power vs. Energy Efficiency**，並補充業界實務中常見的認知誤區與底層機制。

Inference Optimization Mental Map





Precision:

→ The numerical format / bit-width used to represent values.

FP32 → FP16

= reduced precision

Quantization:

→ A technique that maps higher-precision values to a lower-bit representation.

FP32 → INT8

= quantization

1. Inference: GPU or CPU?

Core Answer

Both CPU and GPU are viable for inference. The appropriate choice depends on workload parallelism, latency requirements, throughput targets, model size, concurrency, and deployment environment.

Dimension	GPU	CPU
Architecture	Many highly parallel execution units; SIMT-style programming model	Fewer, more powerful general-purpose cores; strong control-flow capabilities
Main Strength	High throughput for massively parallel workloads	Low latency for small/irregular workloads and flexible general-purpose execution
Typical Use Case	LLMs, multimodal models, CV, high-concurrency server inference	Edge inference, small models, classical ML, low-concurrency serving
Parallelism	Extremely high	Moderate
Memory Bandwidth	Very high; modern accelerators may use HBM	Typically lower; commonly DDR5/LPDDR
Best Scenario	Large models, large batches, high concurrency, matrix-heavy workloads	Small batches, latency-sensitive workloads, edge devices, CPU-only environments

Important: CPU does not always mean low latency, and GPU does not always mean high latency. The actual performance depends on the model, batch size, hardware, implementation, and workload characteristics.

2. Floating-Point Precision and Hardware Design

Core Answer

CPUs and GPUs both support FP32 and FP64, but their hardware is optimized differently. GPUs are particularly strong at massively parallel low-precision matrix operations, while CPUs provide flexible general-purpose computation and strong support for scalar, vector, and control-flow-heavy workloads.

2.1 Numeric Formats

Common numerical formats include:

- **FP64 (Double Precision)**
 - 64-bit floating-point representation
 - High numerical precision
 - Common in scientific and engineering workloads
 - **FP32 (Single Precision)**
 - 32-bit floating-point representation
 - Common in general-purpose ML computation
 - **FP16 (Half Precision)**
 - 16-bit floating-point representation
 - 1 sign + 5 exponent + 10 fraction bits
 - Higher precision than BF16 for a given 16-bit representation
 - Much smaller dynamic range than FP32/BF16
 - Can be more susceptible to overflow/underflow
 - **BF16 (Brain Floating Point)**
 - 16-bit floating-point representation
 - 1 sign + 8 exponent + 7 fraction bits
 - Same exponent width as FP32
 - Much larger dynamic range than FP16
 - Widely used in modern deep-learning training and inference
 - **FP8**
 - 8-bit floating-point representation
 - Used for high-throughput AI workloads
 - Examples include E4M3 and E5M2
 - Supported by modern AI accelerators
 - **INT8 / INT4**
 - Integer-based low-precision representations
 - Commonly used for quantized inference
 - Can significantly reduce model memory footprint and memory bandwidth requirements
-

3. Why Use Lower Precision for Inference?

Lower precision can provide several advantages:

① Lower Memory Footprint

For the same number of parameters:

```
FP32 → 4 bytes / parameter
FP16 → 2 bytes / parameter
INT8 → 1 byte / parameter
INT4 → 0.5 bytes / parameter
```

Therefore, approximately:

```
FP32 → FP16
Model memory ↓ ~50%

FP32 → INT8
Model memory ↓ ~75%

FP32 → INT4
Model memory ↓ ~87.5%
```

This is especially important for large models.

② Lower Memory Bandwidth Requirements

Smaller data types mean fewer bytes need to be moved between:

```
Memory
  ↓
Cache / VRAM
  ↓
Compute Units
```

For memory-bound workloads, reducing data movement can significantly improve inference performance.

③ Hardware Acceleration

Modern CPUs and GPUs provide specialized instructions or hardware units for low-precision computation.

Examples include:

- CPU SIMD/vector instructions

- NVIDIA Tensor Cores
- AMD Matrix Cores
- Specialized INT8/FP8/FP4 acceleration

Therefore:

Lower precision does not automatically mean fewer instructions. The performance benefit comes from a combination of smaller data movement, vectorization, and hardware support for the chosen numerical format.

4. Precision vs. Quantization

These two concepts are related but not identical.

Precision

Precision describes the numerical representation used to store or compute values.

For example:

```
FP32
↓
FP16
```

This is generally described as **reduced precision**.

Quantization

Quantization converts values from a higher-precision representation into a lower-bit representation.

For example:

```
FP32 / FP16
↓
INT8
↓
INT4
```

Quantization often uses a scale and, depending on the scheme, a zero-point or other calibration parameters to approximate the original floating-point values.

The goal is to reduce:

- Model size
- Memory bandwidth
- Memory usage

- Inference cost

while keeping accuracy degradation within an acceptable range.

Precision vs. Quantization — Quick Comparison

Concept	Example	Main Idea
Reduced Precision	FP32 → FP16	Use fewer bits for floating-point representation
Quantization	FP16 → INT8	Map values to a lower-bit representation
Mixed Precision	FP32 + FP16/BF16	Use different precisions for different operations

Common LLM quantization methods include:

- **GPTQ**
- **AWQ**
- **SmoothQuant**
- **Post-Training Quantization (PTQ)**
- **Quantization-Aware Training (QAT)**

5. Does GPU Consume More Power Than CPU?

Core Answer

GPU accelerators often have higher instantaneous power consumption, but higher power does not automatically mean higher energy per inference.

The fundamental relationship is:

$$E = P \times t$$

where:

- E = Energy
- P = Power
- t = Execution time

For example:

```
GPU:
Power ↑
Execution time ↓

CPU:
Power ↓
Execution time ↑
```

The final energy consumption depends on both power and execution time.

Therefore:

A GPU may achieve lower energy per inference if its higher power consumption is offset by substantially shorter execution time, but this is workload- and hardware-dependent.

Other important factors include:

- Batch size
- Model architecture
- Hardware utilization
- Memory bandwidth
- Precision
- Kernel implementation
- CPU/GPU idle power
- Data transfer overhead

TDP

Thermal Design Power (TDP) is a thermal/power design specification and should not be interpreted as the exact energy consumed by one inference.

For example, a GPU rated at 700 W does **not** mean every inference consumes 700 joules.

You need:

$$Energy = Average\ Power \times Execution\ Time$$

6. Memory-Bound vs. Compute-Bound

One of the most important concepts in inference optimization is understanding whether the workload is limited by:

- **Compute**
- **Memory bandwidth**

6.1 Compute-Bound

A workload is **compute-bound** when the hardware spends most of its time performing arithmetic operations.

Increasing compute capability can improve performance.

Typical examples:

Large matrix multiplications
High arithmetic intensity
Large-batch workloads

6.2 Memory-Bound

A workload is **memory-bound** when the bottleneck is moving data rather than performing arithmetic.

In this case:

More FLOPs
↓
does not necessarily
↓
make the workload faster

Increasing memory bandwidth or reducing data movement may help more.

6.3 LLM Prefill vs. Decode

For autoregressive LLM inference, the workload is often divided into:

Prefill

The model processes the input context.

Prompt
↓
Parallel processing
↓
Large matrix operations

Prefill is often relatively **compute-bound**, especially with sufficiently large batches or long prompts.

Decode

The model generates tokens one at a time.

Token 1
↓
Token 2
↓
Token 3



At small batch sizes, decode is often **memory-bound**, because model weights need to be accessed repeatedly while only a relatively small amount of computation is performed per token.

Important: Prefill is not always compute-bound and decode is not always memory-bound. Batch size, sequence length, model architecture, hardware, and implementation can shift the bottleneck.

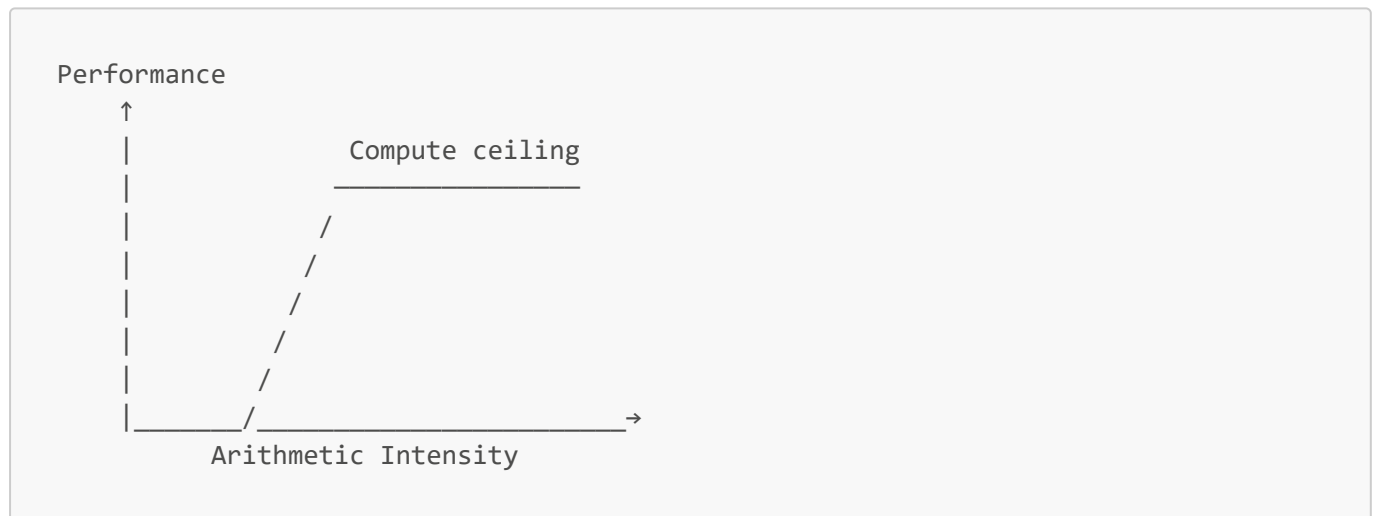
7. Roofline Model

The **Roofline Model** provides a useful way to understand whether a workload is compute-bound or memory-bound.

It relates:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}$$

Conceptually:



If arithmetic intensity is low:

Memory-bound

If arithmetic intensity is high:

Compute-bound

This explains why increasing batch size can sometimes move LLM inference from a memory-bound region toward a compute-bound region.

8. Batch Size and Inference Performance

Batch size has a major effect on inference performance.

Consider a model with a large set of weights:

```
Weight loading
  ↓
Request A
Request B
Request C
Request D
```

With larger batches, the same weights can be reused across more requests.

Therefore:

```
Batch size ↑
  ↓
Weight loading amortized
  ↓
Arithmetic intensity ↑
  ↓
GPU utilization ↑
  ↓
Throughput ↑
```

However:

Larger batch size usually improves throughput but can increase latency.

Therefore, serving systems must balance:

- Latency
- Throughput
- Batch size
- GPU utilization
- Memory usage

9. Unified Memory Blurs the CPU/GPU Boundary

Traditional systems often look like:

```
CPU
  ↓
```

PCIe
↓
GPU
↓
VRAM

Moving large amounts of data between CPU memory and GPU memory can introduce bandwidth and latency overhead.

Some modern architectures reduce this separation.

Examples include:

Apple Silicon

CPU and GPU share a unified memory architecture.

NVIDIA Grace Hopper / Grace Blackwell systems

CPU and GPU are tightly integrated with high-bandwidth memory interconnects and shared/coherent memory mechanisms.

The important idea is:

Modern heterogeneous systems increasingly reduce the cost of communication between CPU and GPU, although CPU and GPU still have distinct compute resources and memory characteristics.

10. Speculative Decoding

Speculative decoding uses a small model to propose tokens and a larger model to verify them.

Conceptually:

```
Small Draft Model
  ↓
Generate candidate tokens
  ↓
Large Target Model
  ↓
Verify candidates in parallel
  ↓
Accept multiple tokens
```

Without speculative decoding:

Large Model



Token 1



Token 2



Token 3

With speculative decoding:

Small Model



Token 1, 2, 3, 4



Large Model verifies



Several tokens may be accepted

The key idea is:

Use inexpensive computation from a small model to reduce the number of expensive sequential decoding steps performed by the large model.

This can reduce latency when the draft model has a sufficiently high acceptance rate.

11. FP8 / FP4 and the Hardware Roadmap

Modern AI hardware increasingly supports lower-precision formats.

NVIDIA Hopper

Supports FP8 through the Transformer Engine.

Common FP8 formats include:

- E4M3
- E5M2

NVIDIA Blackwell

Adds strong support for very-low-precision AI computation, including **FP4/NVFP4**.

The overall trend is:

FP32



```
FP16 / BF16
↓
FP8
↓
FP4 / other low-precision formats
```

The motivation is:

```
Fewer bits
↓
Less memory
↓
Less memory movement
↓
Higher compute density
↓
Higher throughput / lower cost
```

However, lower precision introduces greater numerical error, so hardware and software increasingly use techniques such as:

- Scaling
- Per-tensor scaling
- Per-channel scaling
- Per-group scaling
- Microscaling

to preserve model quality.

12. KV Cache and PagedAttention

During autoregressive generation, the model stores previously computed **Key (K)** and **Value (V)** tensors in the KV cache.

Conceptually:

```
Token 1 → K,V
Token 2 → K,V
Token 3 → K,V
...
Token N → K,V
```

Therefore:

$$KV \text{ Cache Size} \propto \text{Sequence Length}$$

and it also scales with factors such as:

- Number of layers
- Number of KV heads
- Head dimension
- Batch size
- Number of concurrent requests

For long-context and high-concurrency LLM serving, KV cache can become a **major or even dominant consumer of GPU memory**.

PagedAttention

Traditional memory allocation can cause fragmentation.

PagedAttention, popularized by vLLM, manages KV cache using smaller blocks/pages rather than requiring one large contiguous allocation.

Conceptually:

```
Traditional:  
Request A → [████████████████████]  
  
Request B → [██████████]  
  
Free space → fragmented
```

Paged approach:

```
Page 1 → Request A  
Page 2 → Request B  
Page 3 → Request A  
Page 4 → Request C  
...
```

This improves memory utilization and allows serving systems to handle dynamic workloads more efficiently.

13. Continuous / In-Flight Batching

Static batching can be inefficient when requests have different sequence lengths.

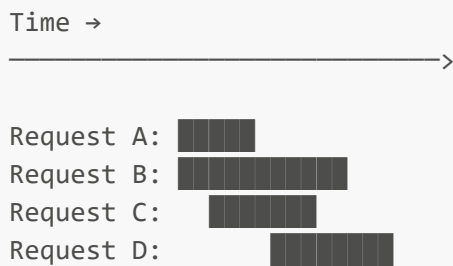
For example:

```
Request A → finishes quickly  
Request B → finishes slowly  
Request C → finishes slowly
```

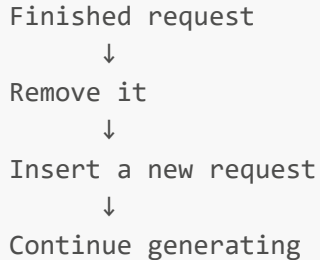
With static batching, the system may need to wait for the entire batch.

Continuous batching, also called **in-flight batching**, dynamically adds and removes requests during generation.

Conceptually:



Instead of waiting for an entire batch to finish:



This helps maintain high accelerator utilization under variable-length workloads.

Continuous batching is commonly used in modern LLM serving systems such as:

- vLLM
- TensorRT-LLM
- Hugging Face TGI

14. Precision, Quantization, and CPU Inference

This connects directly to an important inference optimization question:

Can a model be converted to a lower precision and then run on CPU?

Yes.

For example:

```
FP32 Model
  ↓
Quantization
  ↓
INT8 / INT4 Model
  ↓
CPU Inference
```

Potential benefits include:

- Smaller model size
- Lower memory usage
- Lower memory bandwidth requirements
- Better cache utilization
- Vectorized INT8/INT4 computation on supported CPUs

However:

Quantization does not automatically mean fewer CPU instructions.

Performance depends on whether the CPU has efficient instructions and libraries for the chosen data type.

For example:

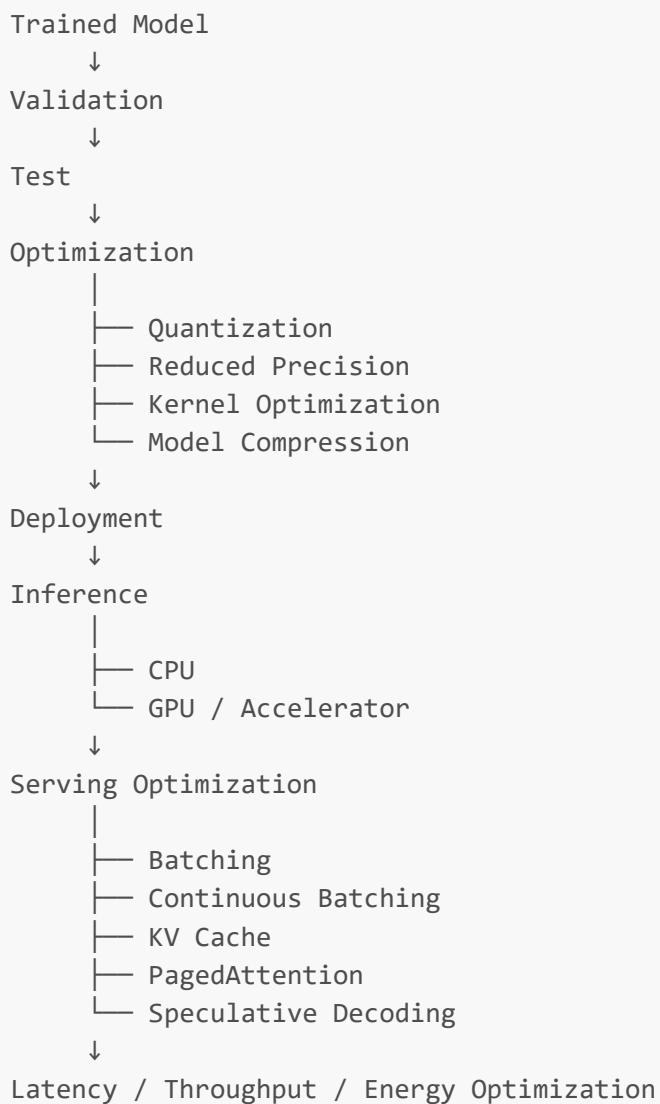
```
INT8
  ↓
SIMD / Vector Instructions
  ↓
Process many values per instruction
  ↓
Higher throughput
```

Therefore, the real optimization comes from a combination of:

```
Lower precision
  +
Less memory movement
  +
Vectorization
  +
Hardware-specific instructions
  +
Optimized inference kernels
```

15. End-to-End Inference Pipeline

The concepts above can be connected into a complete inference pipeline:



16. Key Takeaways

Hardware

- CPU
 - Flexible
 - Strong control flow
 - Good for small/irregular workloads
- GPU
 - Massive parallelism

- High throughput
- Excellent for matrix-heavy workloads

Precision

```
FP64
↓
FP32
↓
FP16 / BF16
↓
FP8
↓
FP4 / INT8 / INT4
```

Lower precision can provide:

```
Memory ↓
Bandwidth requirement ↓
Compute density ↑
Potential throughput ↑
```

but may introduce numerical error.

Quantization

```
High precision
↓
Lower-bit representation
↓
Smaller model
↓
Less memory traffic
↓
Potentially faster / cheaper inference
```

LLM Inference

```
Prefill
→ often compute-bound

Decode
→ often memory-bound at small batch sizes
```

Batch size ↑
→ weight reuse ↑
→ arithmetic intensity ↑
→ throughput ↑

Energy

Power ≠ Energy

Energy = Power × Time

A GPU can consume more instantaneous power but potentially use less energy per inference if it completes the workload much faster. This must be measured for the actual workload.

Serving

KV Cache
→ memory management

PagedAttention
→ efficient KV-cache allocation

Continuous Batching
→ higher utilization

Speculative Decoding
→ reduce sequential decoding latency

Suggested Next Additions

If you want to go deeper, the next useful topics are:

1. Numerical stability in mixed-precision training

- Loss scaling
- FP16 vs. BF16
- Gradient underflow / overflow

2. MoE (Mixture-of-Experts) inference

- Expert routing
- Memory vs. compute tradeoffs
- Communication overhead

3. LLM serving engine comparison

- vLLM
- TensorRT-LLM
- SGLang

4. Distributed inference

- Tensor Parallelism
- Pipeline Parallelism
- Expert Parallelism
- Data Parallelism

5. Performance metrics

- TTFT (Time to First Token)
- TPOT (Time Per Output Token)
- Tokens/sec
- Requests/sec
- Joules/token

6. End-to-end inference optimization

- Model → Precision → Kernel → Memory → Batching → Serving → Hardware